



Institute of
Emerging
Technologies



Institute of
Emerging
Technologies

COMPILER CONSTRUCTION

SIR ABDUL REHMAN QURESHI

NAME: MUREED ABBAS

CLASS: BSCS V

SECTION: A

CMS ID : 023-24-0439

MUREED

LAB : Three



Institute of
Emerging
Technologies

1. Introduction

This lab focuses on lexical analysis and regular expressions. A lexical analyzer, also called a scanner, is the first phase of a compiler. It reads source-code characters, groups them into lexemes, and produces tokens. Regular expressions describe the patterns of common tokens such as identifiers, numbers, keywords, and operators.

In this lab, I wrote regular expressions for different token descriptions, tested identifiers using Python's `re.fullmatch()`, and implemented a small rule-based lexical analyzer. The lexer was then extended to recognize keywords, relational operators, comments, and line numbers.

2. Exercise 1 — Write the Regex

Q1. Identifier starting with a lowercase letter:

```
[a-z][A-Za-z0-9]*
```

Examples: hello, student1,

abc123

Q2. Binary number:

```
[01]+
```



String	Result	Reason
Lab4	Accepted	Starts with a letter and is followed by letters/digits.

4Lab	Rejected	Starts with a digit; the first character must be a letter.
total_sum	Rejected	Underscore is not allowed by the given pattern.

Examples: 0, 1, 101, 11001

Q3. Valid C variable name:

`[A-Za-z_][A-Za-z0-9_]*`

Examples: name, _total, student1, student_name

Q4. Email-like username:

`[a-z]+\.[a-z]+`

Examples: faryal.shamsi, ali.khan, saifullah.ramzan

Q5. Comment between `/*` and `*/`:

`/\*(?:[^\*]|\*(?!/))*\*/`

The expression matches `/* ... */` without allowing another `*/` inside.

3. Exercise 2 — Trace the Regex

Given regex:

`[A-Za-z][A-Za-z0-9]*`

"" (empty string)	Rejected	The pattern requires a starting letter.
-------------------	----------	---

Exercise 3 — Test with Python

```
--- EXERCISE 3 ---
student -> Valid Identifier
_name -> Valid Identifier
student123 -> Valid Identifier
123student -> Invalid Identifier
total-name -> Invalid Identifier
PS D:\5th Semester\Compiler Construction\lab> █
```

Exercise 4 — Extend the Mini Lexer

```
--- EXERCISE 4 ---
('KEYWORD', 'if')
('LPAREN', '(')
('ID', 'x')
('REL_OP', '<=')
('NUMBER', '10')
('RPAREN', ')')
('DIVIDE', '/')
('DIVIDE', '/')
('ID', 'check')
('ID', 'limit')
('ID', 'y')
('ASSIGN', '=')
('ID', 'x')
('PLUS', '+')
('NUMBER', '1')
PS D:\5th Semester\Compiler Construction\lab> █
```

Exercise 5 — Challenge

--- EXERCISE 5 ---

('KEYWORD', 'if', 1)

('LPAREN', '(', 1)

('ID', 'x', 1)

('REL_OP', '<=', 1)

('NUMBER', '10', 1)

('RPAREN', ')', 1)

('ID', 'y', 2)

('ASSIGN', '=', 2)

('ID', 'x', 2)

('PLUS', '+', 2)

('NUMBER', '1', 2)

('KEYWORD', 'while', 3)

('LPAREN', '(', 3)

('ID', 'y', 3)

('REL_OP', '>', 3)

('NUMBER', '0', 3)

('RPAREN', ')', 3)

PS D:\5th Semester\Compiler Construction\lab> █